

Tanya

Polymath Jr

07/12/2025

### • Optimal Transport (Monge formulation)

source distribution  $\rho$ , target dist.  $\mu$

find the map  $T$  s.t.

$$\min C(T) = \int \underbrace{c(x, T(x))}_{\text{Eg. quadratic cost}} \rho(x) dx \quad \text{s.t. } T\#\rho = \mu$$

Eg. quadratic cost  $c(x, y) = \frac{1}{2} \|x - y\|^2$

## KL Divergence vs. Optimal Transport

There are two major paradigms for learning transport maps from a source distribution  $\rho$  to a target distribution  $\mu$ :

### Optimal Transport (Cost-Based Formulation)

In the classical optimal transport (OT) setup, we are given a cost function  $c(x, y)$  — for example, quadratic cost:

$$c(x, y) = \frac{1}{2} \|x - y\|^2$$

We then seek a transport map  $T$  that pushes  $\rho$  onto  $\mu$  while minimizing the expected cost:

$$\min_T \int c(x, T(x)) d\rho(x) \quad \text{subject to } T\#\rho = \mu$$

This approach gives the **most efficient** way to move probability mass from  $\rho$  to  $\mu$ . If multiple maps  $T$  satisfy the pushforward constraint  $T\#\rho = \mu$ , then the cost function picks the **optimal one**.

### KL Gradient Flow (Divergence-Based Formulation)

Instead of defining a cost, here we minimize the Kullback–Leibler divergence:

$$\min_{\rho_t} \text{KL}(\rho_t \| \mu)$$

This leads to a gradient flow on the space of probability measures. The evolution of particles  $z(t)$  under this flow is governed by:

$$\dot{z}(t) = -\nabla_z (\log \rho_t(z) - \log \mu(z))$$

This approach does not rely on a transport cost. Instead, it continuously morphs  $\rho_t$  into  $\mu$  by following the steepest descent direction in KL divergence.

## Connection and Key Differences

- Both methods construct a flow from  $\rho$  to  $\mu$ .
- **OT** uses a cost function  $c(x, y)$  to select the best map among all that satisfy  $T_{\#}\rho = \mu$ .
- **KL gradient flow** only cares about reducing divergence — it doesn't optimize cost.

Lagrange  $\Rightarrow$  Let  $z := T(x, t)$  (map/flow at time  $t$ )  
 Dual  
 particle flow  $\left\{ \begin{array}{l} \dot{z} = -\nabla_z \left[ \frac{d\tilde{L}_t}{du} \right] \\ z(t=0) = x \end{array} \right.$   
 (Gradient Flow)  
 $u = \frac{\|z\|^2}{2} \Rightarrow \frac{d\tilde{L}_t}{du} = -\nabla_z (\rho_t(z) - \mu(z))$   
 or  
 at time  $t$ ,  $\{z_i\} \sim \rho_t$

**Note:** This is a **particle flow**, not a normalizing flow. Therefore, there is *no parameterization* of the map  $T$ ; we directly evolve particles over time rather than learning a parametric function.

Suppose we are given:

- A **source distribution**  $\rho_0$ , which we have access to through a set of samples  $\{x_i\}_{i=1}^N \subset \mathbb{R}^d$ . This is our starting point.
- A **target distribution**  $\mu$ , which we cannot sample from, but whose probability density function is known.

We want to transform particles  $x \sim \rho_0$  into particles  $z \sim \mu$ . To do this, we define a time-dependent flow map:

$$z(t) := T(x, t),$$

which describes how each particle evolves over time. The initial condition is:

$$T(x, 0) = x.$$

## Gradient Flow in Probability Space

Let  $\rho_t$  denote the distribution of the particles  $z(t)$  at time  $t$ . We define a loss functional that measures how far  $\rho_t$  is from the target distribution  $\mu$ . A common choice is the **Kullback-Leibler (KL) divergence**:

$$\mathcal{L}[\rho_t] := \text{KL}(\rho_t \parallel \mu) = \int \rho_t(z) \log \left( \frac{\rho_t(z)}{\mu(z)} \right) dz.$$

We want to evolve the particles along the direction that most steeply decreases this loss. This defines a **gradient flow** in the space of distributions.

## Lagrangian (Particle-Based) Formulation

Instead of describing how the entire probability distribution  $\rho_t$  evolves over time (Eulerian view), we focus on how individual particles move (Lagrangian view).

Let  $z(t) \in \mathbb{R}^d$  denote the position of a particle at time  $t$ , where each particle starts at some initial point  $x \sim \rho_0$ .

Our goal is to evolve these particles so that their collective distribution  $\rho_t$  moves closer to the target distribution  $\mu$  over time.

To achieve this, we define the dynamics of each particle using a differential equation derived from minimizing the KL divergence  $\text{KL}(\rho_t \parallel \mu)$ . This gives us the following update rule for the particle trajectories:

$$\frac{d}{dt}z(t) = -\nabla_z \left( \frac{\delta \mathcal{L}}{\delta \rho_t}(z) \right) = -\nabla_z \log \left( \frac{\rho_t(z)}{\mu(z)} \right) = \nabla_z \log \mu(z) - \nabla_z \log \rho_t(z).$$

$$z^{(t+\Delta t)} = z^{(t)} + \Delta t \cdot \left( \nabla_z \log \mu(z^{(t)}) - \nabla_z \log \rho_t(z^{(t)}) \right)$$

- $\frac{d}{dt}z(t)$  is the velocity of the particle — how it moves over time.
- $\mu(z)$  is the target density — what we want the particle distribution to become.
- $\rho_t(z)$  is the current density of the particles at time  $t$ .

The **functional derivative** tells you how the loss changes if you change the density at a specific location  $z$ . The gradient of the functional derivative points in the direction that increases the rate at which the loss changes if you perturb the density at that location. We move in the opposite direction to decrease it.

$$\left\langle \frac{\partial}{\partial z_1} \left( \frac{\delta \mathcal{L}}{\delta \rho}(z) \right), \frac{\partial}{\partial z_2} \left( \frac{\delta \mathcal{L}}{\delta \rho}(z) \right), \dots, \frac{\partial}{\partial z_d} \left( \frac{\delta \mathcal{L}}{\delta \rho}(z) \right) \right\rangle$$

• Data-Driven case :  $\{x_i\}_{i=1}^N \sim \rho, \{y_j\}_{j=1}^M \sim \mu$

$z_i = x_i$

$T(z_i, N) \sim \mu$

↓  
large time step

$\Rightarrow$  Gradient Descent :  $z_i^{n+1} = z_i^n - \eta \cdot \nabla_z (\rho_t(z) - \mu(z)) \Big|_{z_i^n}$

(Discretize flow)

We follow the negative gradient direction to minimize the objective function (the quantity inside the parentheses).

learning rate

bar means evaluated yet

$\mu(z_i^n)$

## Two Common Settings: Sampling vs Data-Driven

There are two common settings when working with particle-based flows or variational inference:

## Sampling-only (density-based) setup

- You only have samples from  $\rho$  (e.g., Gaussian noise).
- You know only the form of  $\mu$  (e.g., an unnormalized log-density, such as in energy-based models).
- This is the most common case in variational inference and score-based generative modeling.

## Data-driven setup

- You have access to samples from both  $\{x_i\} \sim \rho$  (base samples) and  $\{y_j\} \sim \mu$  (target samples).
- You use these samples to estimate the density mismatch  $\rho_t(z) - \mu(z)$  empirically, often via kernel density estimation or score matching.

## Gradient vs Partial Derivative with Respect to a Vector

In this context, the operators

$$\frac{\partial}{\partial z_i} \quad \text{and} \quad \nabla_{z_i}$$

mean the same thing: they both refer to the gradient with respect to the vector  $z_i \in \mathbb{R}^d$ .

Function is defined on second to last line of Tanya's notes

learning rate

$$\textcircled{1} \text{ KDE : } \nabla_z (\rho_t(z) - \mu(z)) \approx \frac{1}{N} \sum_{i=1}^N \nabla_z K_h(z, z_i^n) - \frac{1}{M} \sum_{j=1}^M \nabla_z K_h(z, y_j)$$

$\{z_i\}_{i=1}^N \sim \rho_t$     $\{y_j\}_{j=1}^M \sim \mu$    Kernel matrix  $K(z_i^n, z_j^n) \Big|_{\substack{i=1, \dots, N \\ j=1, \dots, M}}$

$K(z_i^n, y_j) \Big|_{\substack{i=1, \dots, N \\ j=1, \dots, M}}$

$$\textcircled{2} \quad \rho_t(z) - \mu(z) = \int \delta(z - z') (\rho_t(z') - \mu(z')) dz'$$

Different formula approach

$$\approx \int \underbrace{K(z, z')}_{\text{"local window"}} (\rho_t(z') - \mu(z')) dz'$$

$\delta(t) = \begin{cases} 1, & t=0 \\ 0, & \text{o/w} \end{cases}$

$$= \int K(z, z') \rho_t(z') dz' - \int K(z, z') \mu(z') dz'$$

Monte Carlo

$$\approx \frac{1}{N} \sum_i K(z, z_i^n) - \frac{1}{M} \sum_j K(z, y_j)$$

# From Dirac Delta to Monte Carlo

We are interested in estimating the difference between two probability density functions at a given point  $z$ :

$$\rho_t(z) - \mu(z),$$

where  $\rho_t(z)$  is the model or generated density at time  $t$ , and  $\mu(z)$  is the reference or target density.

## Dirac Delta Reformulation

We begin by expressing the pointwise difference using the Dirac delta function:

$$\rho_t(z) - \mu(z) = \int \delta(z - z') [\rho_t(z') - \mu(z')] dz'.$$

This holds because the Dirac delta  $\delta(z - z')$  satisfies the property:

$$\int \delta(z - z') f(z') dz' = f(z),$$

which allows us to recover the function value at  $z$  through integration.

## Kernel Approximation of the Dirac Delta

The Dirac delta is not computationally tractable due to its singular nature, so we approximate it using a smooth kernel function  $k(z, z')$ . This yields:

$$\rho_t(z) - \mu(z) \approx \int k(z, z') [\rho_t(z') - \mu(z')] dz'.$$

Here,  $k(z, z')$  serves as a localized weighting function (or “local window”) centered at  $z$ , enabling us to compare how much mass each distribution places near  $z$  rather than exactly at  $z$ .

## Understanding the Area Under the Dirac Delta Approximation

To build intuition for the Dirac delta function  $\delta(z - z')$ , we start with a simpler, tractable approximation using a rectangle function. Define:

$$\delta_\epsilon(z - z') = \begin{cases} \frac{1}{\epsilon}, & \text{if } |z - z'| \leq \frac{\epsilon}{2}, \\ 0, & \text{otherwise} \end{cases}$$

This function represents a rectangle with the following properties:

- **Width:**  $\epsilon$
- **Height:**  $\frac{1}{\epsilon}$
- **Centered at:**  $z = z'$

Now compute the **area under the curve**:

$$\text{Area} = \text{width} \times \text{height} = \epsilon \cdot \frac{1}{\epsilon} = 1$$

That’s it — the total area under  $\delta_\epsilon(z - z')$  is 1. This is the key property that makes  $\delta_\epsilon(z - z')$  a valid approximation to the Dirac delta in the limit  $\epsilon \rightarrow 0$ . As the rectangle becomes infinitely narrow and infinitely tall, the area remains constant at 1.

## Splitting the Integral

The above expression can be separated into two integrals:

$$\int k(z, z') \rho_t(z') dz' - \int k(z, z') \mu(z') dz'.$$

Each term represents a kernel-weighted average of the respective density around the point  $z$ .

## Monte Carlo Approximation

We now estimate each integral using empirical samples:

- Let  $\{z_i^{\sim}\}_{i=1}^N \sim \rho_t$  be i.i.d. samples from the model density.
- Let  $\{y_j\}_{j=1}^M \sim \mu$  be i.i.d. samples from the reference density.

Using these samples, we approximate:

$$\int k(z, z') \rho_t(z') dz' \approx \frac{1}{N} \sum_{i=1}^N k(z, z_i^{\sim}), \quad \int k(z, z') \mu(z') dz' \approx \frac{1}{M} \sum_{j=1}^M k(z, y_j).$$

## Final Expression

Combining the two Monte Carlo estimates, we obtain the final approximation:

$$\rho_t(z) - \mu(z) \approx \frac{1}{N} \sum_{i=1}^N k(z, z_i^{\sim}) - \frac{1}{M} \sum_{j=1}^M k(z, y_j).$$

• KDE (Kernel Density Estimate)

- Non-parametric

• Parametric:  $\mathcal{P} = \{P_\theta : \theta \in \Theta\}, \Theta \subseteq \mathbb{R}^p$  (finite dim)

- Eg.  $N(\underset{\uparrow}{\mu}, \underset{\uparrow}{\sigma^2})$ ,  $p=2$

Now,  $\mathcal{P} = \{ \text{all cts distribution on } \mathbb{R} \}$  (1D)

$= \{f: \mathbb{R} \rightarrow \mathbb{R} \mid f \geq 0, \int f = 1\}$

Question: Can we estimate  $f$  (based on data/stats) ?

(Under smoothness assumption for  $f$ )

# Kernel Density Estimation (KDE): Parametric vs. Non-Parametric

## Parametric Density Estimation

In a parametric framework, we assume that the unknown data-generating distribution belongs to a family of distributions that can be fully described by a finite-dimensional parameter vector.

- Let  $\theta \in \Theta \subset \mathbb{R}^p$  be a parameter vector.
- Let  $P_\theta$  denote a probability distribution indexed by  $\theta$ .
- The parametric model class is defined as:

$$\mathcal{P}_{\text{param}} = \{P_\theta : \theta \in \Theta\}.$$

- This family is finite-dimensional because the model is fully described by the  $p$ -dimensional vector  $\theta$ .

**Example:** The family of univariate Gaussian distributions:

$$P_\theta = \mathcal{N}(\mu, \sigma^2), \quad \theta = (\mu, \sigma^2) \in \mathbb{R}, \mathbb{R}_{>0}.$$

Here, the distribution is fully specified by two parameters, so  $p = 2$ .

## Non-Parametric Density Estimation

In contrast, non-parametric models do not assume a fixed finite-dimensional form for the distribution. Instead, the density is allowed to lie in an infinite-dimensional function space.

- Let  $f : \mathbb{R} \rightarrow \mathbb{R}$  be a probability density function (pdf).
- The non-parametric model class is:

$$\mathcal{P}_{\text{non-param}} = \left\{ f : \mathbb{R} \rightarrow \mathbb{R} \mid f(x) \geq 0, \int_{\mathbb{R}} f(x) dx = 1 \right\}.$$

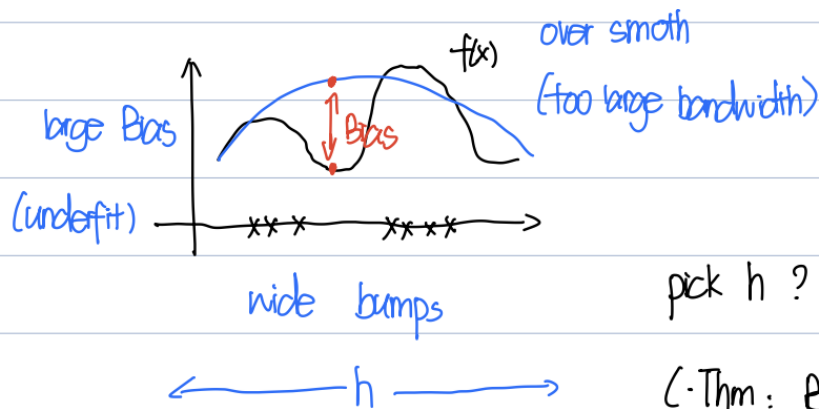
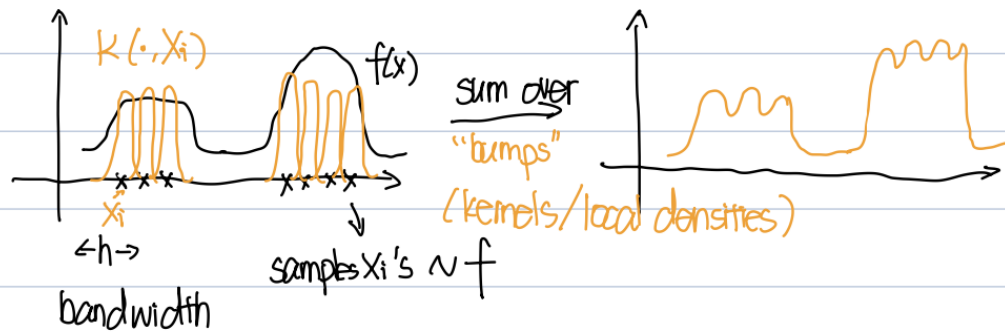
- This class includes all continuous, integrable functions that integrate to one.

## Key Question

**Goal:** Can we estimate the unknown density  $f \in \mathcal{P}_{\text{non-param}}$  using a finite set of data samples?

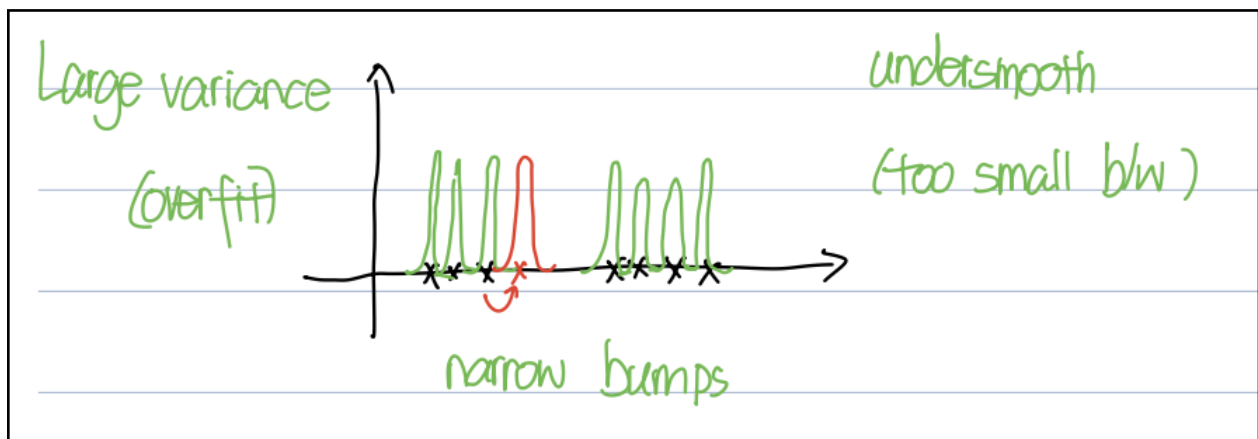
This leads to kernel density estimation (KDE), which constructs an estimator  $\hat{f}$  based on observed data, under the assumption that the true density  $f$  is sufficiently smooth.

## • KDE



pick  $h$  ? Eg. Rule of Thumb

(-Thm: Bias-Variance Trade off  
 $\Rightarrow h \sim n^{-1/3}$  )



## Kernel Density Estimation (KDE)

This section explains how Kernel Density Estimation (KDE) works and how the bandwidth parameter  $h$  affects the resulting density estimate.



## Overview of KDE

Suppose we are given a set of data samples  $x_1, x_2, \dots, x_n \sim f$ , drawn from some unknown density  $f$ . KDE constructs an estimate of  $f$  by placing a smooth kernel function  $K$  centered at each data point. The density estimate is defined as:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

where:

- $K$  is a kernel function (e.g., Gaussian).
- $h > 0$  is the bandwidth, controlling the width of each kernel bump.

## Effect of Bandwidth $h$

The bandwidth  $h$  plays a critical role in the quality of the density estimate:

- If  $h$  is too large, the estimate becomes overly smooth. This results in high **bias** because important features of  $f$  are averaged out.
- If  $h$  is too small, the estimate is too sensitive to noise. This results in high **variance** because small fluctuations in the data dominate the estimate.

## Bandwidth Selection

Choosing  $h$  optimally balances bias and variance. A common rule-of-thumb suggests:

$$h \propto n^{-1/5} \quad (\text{in 1D})$$

$$h \propto n^{-1/(d+4)} \quad (\text{in } d\text{-dimensions})$$

In the figure provided, the heuristic  $h \sim n^{-1/3}$  is shown, which may arise under specific assumptions about smoothness or kernel shape.

• A kernel is a func.  $K: \mathbb{R} \rightarrow \mathbb{R}$

①  $\int K(u) du = 1$

②  $\int u K(u) du = 0$   $\Rightarrow K_h(\cdot, z) \sim N(z, h^2)$

-Eg. (1)  $K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}$

0)  $K(u) = \frac{1}{2} \cdot \mathbb{1}_{[-1,1]}$

The figure contains two hand-drawn graphs. The top graph shows a Gaussian kernel centered at 0, with a bell-shaped curve. The bottom graph shows a uniform kernel on the interval [-1, 1], with a rectangular pulse of height 1/2.

## Interpretation of Equation (2): Centering Condition

Equation (2) from the kernel definition is:

$$\int u k(u) du = 0$$

This condition means that the first moment of the kernel function  $k(u)$  is zero:

$$\mathbb{E}_{U \sim k}[U] = 0.$$

## Why the Expectation Represents the Center of a Distribution

The expectation of a random variable  $X$  with density  $f(x)$  is defined as:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x) dx$$

This is the **weighted average** of all possible values of  $X$ , using the probability density as weights. It represents the **center of mass** of the distribution — the point where the distribution would balance if it were a physical object.

If  $\mathbb{E}[X] = 0$ , this means that the average value of the distribution is zero. The contributions from negative and positive values of  $X$  perfectly cancel out in expectation. In this case, we say the distribution is **centered at 0**.

### Clarifications:

- The expectation  $\mathbb{E}[X]$  is always the center of mass, regardless of the shape of the distribution.
- The median is the point that divides the total probability in half:

$$\mathbb{P}(X \leq \text{median}) = \mathbb{P}(X \geq \text{median}) = 0.5$$

- The median is always the center in terms of total probability mass, not necessarily in terms of average value.
- When the distribution is symmetric, the mean, median, and mode are all equal:

$$\mathbb{E}[X] = \text{median} = \text{mode}$$

- In a right-skewed distribution, the mean is typically greater than the median and mode:

$$\text{mode} < \text{median} < \mathbb{E}[X]$$

- In a left-skewed distribution, the mean is typically less than the median and mode:

$$\mathbb{E}[X] < \text{median} < \text{mode}$$

We will consider for  $h > 0$ ,  $z \in \mathbb{R}$   
 $\hookrightarrow$  bandwidth

$\frac{1}{h} k\left(\frac{\bullet - z}{h}\right)$  rescaled version of  $k$  centered at  $z$

$X_1, \dots, X_n \sim$  from unknown dist.

our estimator  $\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n k\left(\frac{x - X_i}{h}\right) = \frac{1}{n} \sum_{i=1}^n k_h(x, X_i)$   
 $\uparrow$  normalizing factor  $\Rightarrow \int \hat{f}(x) dx = 1$

## Understanding the Kernel Density Estimator (KDE)

Given data points  $X_1, \dots, X_n \sim$  from an unknown distribution, our goal is to estimate the probability density function  $f(x)$  — that is, how likely values are to occur near each point  $x \in \mathbb{R}$ .

We use the kernel density estimator:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n k\left(\frac{x - X_i}{h}\right)$$

This formula constructs  $\hat{f}(x)$  by placing a smooth "bump" (the kernel function  $k$ ) centered at each data point  $X_i$ , then averaging their contributions at location  $x$ . Here's how to interpret each part.

### The expression $x - X_i$

This measures the distance between the evaluation point  $x$  and the data point  $X_i$ . When  $x = X_i$ , this term is zero — the kernel is centered at its peak. As  $x$  moves away from  $X_i$ , the kernel value decreases.

### Dividing by $h$

The bandwidth parameter  $h > 0$  controls how wide or narrow the kernel is. Inside the kernel, we compute:

$$\frac{x - X_i}{h}$$

This rescales the horizontal axis. If  $h$  is small, the kernel is sharply peaked around  $X_i$ . If  $h$  is large, the kernel is wide and smooth. This determines how much influence each point  $X_i$  has in the neighborhood of  $x$ .

## The outer factor $\frac{1}{h}$

When we squeeze a kernel horizontally by a factor of  $h$ , its area shrinks. To keep the area under the curve equal to 1 (so that it's a proper probability density), we multiply vertically by  $\frac{1}{h}$ . This normalization ensures:

$$\int \frac{1}{h} k\left(\frac{x - X_i}{h}\right) dx = 1$$

## Summing over $i = 1$ to $n$

Each data point  $X_i$  contributes one kernel bump centered at its location. The sum:

$$\sum_{i=1}^n \frac{1}{h} k\left(\frac{x - X_i}{h}\right)$$

adds up the contributions from all points.

## Dividing by $n$

We divide by  $n$  to take the *average* contribution from all data points. This makes  $\hat{f}(x)$  behave like an empirical estimate of the true density — much like a histogram averages the frequency of observations in a bin.

## Interpretation

- For each point  $X_i$ , we place a rescaled kernel centered at  $X_i$ .
- These kernels are wider or narrower depending on the bandwidth  $h$ .
- The KDE  $\hat{f}(x)$  is constructed by summing all those bumps and averaging them.
- This results in a smooth estimate of the unknown density.

## Why the KDE Average Makes Sense

A natural question arises when interpreting the KDE formula:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n k\left(\frac{x - X_i}{h}\right)$$

At any location  $x$ , we average the contributions from all  $n$  kernels. But since most of the data points  $X_i$  are far from  $x$ , their kernels contribute almost nothing. So how can this average possibly reflect high density near  $x$ , especially when only one or two nearby points contribute meaningfully?

**Answer:** This behavior is intentional — it's exactly what makes KDE work correctly.

- The kernel  $k\left(\frac{x - X_i}{h}\right)$  is large only when  $x$  is close to  $X_i$ . Otherwise, it's close to zero.
- When evaluating  $\hat{f}(x)$ , most terms in the sum are nearly zero unless the corresponding  $X_i$  is close to  $x$ .
- So the KDE effectively counts how many points are near  $x$ , and how close they are.
- If few data points are near  $x$ , then the sum will be small, and the average will be small — indicating low density. If many data points are close, the kernels will overlap and stack, and the average will be large — indicating high density.
- This averaging reflects the idea that we are estimating the density *locally*, not just checking for exact matches.

## Key insight

The KDE doesn't ask, "Is there a point exactly at  $x$ ?" It asks, "*How much probability mass is near  $x$ ?*" Averaging the kernel contributions is how we answer that. A high value of  $\hat{f}(x)$  means many data points are close to  $x$ , or that those which are nearby give strong support via the kernel.